



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 4

Parpadeo del LED Integrado a la Raspberry Pi Pico W Usando C

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 19 DE MAYO DE 2026

Parpadeo del LED Integrado a la Raspberry Pi Pico W

Mediante Programación en Lenguaje C

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 19 de mayo de 2026

Resumen—Se presenta el desarrollo completo de la Práctica 4, cuyo objetivo es la programación del LED integrado en la placa Raspberry Pi Pico W mediante el lenguaje C y el SDK oficial de Raspberry Pi. Se configuró el entorno de desarrollo utilizando Pico Developer PowerShell y Visual Studio Code, se creó el proyecto con su archivo `CMakeLists.txt` correspondiente, y se implementó un programa que controla el encendido y apagado periódico del LED con la función `cyw43_arch_gpio_put`. El binario compilado en formato `.uf2` fue transferido al microcontrolador, verificando el correcto funcionamiento del parpadeo en intervalos de 250 ms.

en RP2040 que utilicen soporte del SDK.

- Integración con CMake: facilita la configuración y compilación de proyectos de manera estructurada y reproducible.
- Soporte para el chip CYW43439: en la versión Pico W, el SDK incluye librerías dedicadas para controlar el módulo Wi-Fi/Bluetooth y los GPIO asociados a él, como el LED integrado.

I. INTRODUCCIÓN

I-A Objetivo del ejercicio

El objetivo principal de esta práctica es aprender a crear un proyecto desde cero capaz de programar la Raspberry Pi Pico W usando como referencia el código de ejemplo que permite el parpadeo del LED integrado en la placa. Para lograrlo, se utiliza el SDK oficial de Raspberry Pi junto con las librerías `pico_stdlib` y `pico_cyw43_arch_none`.

Durante la práctica se exploró la estructura básica de un proyecto para Raspberry Pi Pico W y la interacción entre el SDK y el hardware integrado.

I-B Importancia del SDK de Raspberry Pi

El SDK (*Software Development Kit*) de la Raspberry Pi proporciona un conjunto de herramientas, librerías y ejemplos que simplifican el desarrollo de aplicaciones embebidas para la familia de microcontroladores RP2040. Entre sus ventajas principales destacan:

- Abstracción del hardware: permite controlar periféricos como GPIO, UART, SPI e I2C sin escribir código de bajo nivel para cada registro.
- Portabilidad: los proyectos compilados con el SDK son compatibles con la mayoría de placas basadas

I-C Aplicaciones del Raspberry Pi Pico W en microcontroladores

La Raspberry Pi Pico W, basada en el microcontrolador RP2040, se posiciona como una plataforma versátil para aplicaciones de IoT (*Internet of Things*), automatización y sistemas embebidos. Algunas de sus aplicaciones más relevantes en el ámbito de la ingeniería mecatrónica incluyen el control de actuadores y sensores en tiempo real, la comunicación inalámbrica mediante Wi-Fi o Bluetooth, el procesamiento de señales analógicas gracias a su ADC de 12 bits, y la implementación de protocolos de comunicación industrial como UART, SPI e I2C. La programación en C con el SDK permite aprovechar al máximo el rendimiento del núcleo Cortex-M0+ a 133 MHz, siendo ideal para tareas de control de precisión.

II. DESARROLLO

II-A Configuración del Entorno de Desarrollo

II-A1. Instalación y configuración del SDK

El primer paso para el desarrollo del proyecto consiste en la instalación del SDK de Raspberry Pi en el equipo de trabajo. Este SDK se descarga desde el repositorio oficial y se instala junto con las herramientas de compilación cruzada (*arm-none-eabi-gcc*) y CMake. Una vez instalado, el sistema crea automáticamente las variables de entorno necesarias, tales como `PICO_SDK_PATH` y `PICO_EXAMPLES_PATH`, que son utilizadas posteriormente en los comandos de configuración del proyecto.

Para comenzar el trabajo, se abre la aplicación **Pico Developer PowerShell** con privilegios de administrador. Esta terminal tiene preconfiguradas todas las rutas y variables de entorno requeridas por el SDK, lo que simplifica significativamente el proceso de desarrollo.

II-A2. Descripción de las librerías utilizadas

pico_stdlib: Es la librería estándar del SDK de Raspberry Pi. Incluye funciones básicas de entrada/salida, temporización (`sleep_ms`, `sleep_us`), control de GPIO estándar, y soporte para comunicación USB y UART. Es el punto de partida obligatorio en cualquier proyecto para la Pico.

pico_cyw43_arch_none: Esta librería gestiona la inicialización y control del chip CYW43439, que es el módulo inalámbrico integrado en la Raspberry Pi Pico W. Dado que en la versión Pico W el LED integrado está conectado a un GPIO del chip CYW43439 (y no directamente al RP2040), es indispensable usar esta librería para poder controlarlo. La variante `none` indica que no se inicializa ningún stack de red, lo que resulta adecuado cuando solo se necesita acceder al LED sin funcionalidad Wi-Fi.

II-A3. Creación de la estructura de carpetas y archivos

Se crea una carpeta principal llamada **MICROCONTROLADORES** para alojar todas las prácticas del curso. Dentro de ella, se genera la carpeta **Práctica_4** (sin espacios en blanco). A continuación, en la terminal Pico Developer PowerShell, se navega a dicha carpeta con el comando `cd` y se copian los archivos base del SDK ejecutando:

COMANDOS POWERSHELL – CONFIGURACIÓN INICIAL

```
cd C:\Users\Carlos\MICROCONTROLADORES\
  Practica_4

copy ${env:PICO_SDK_PATH}\external\
  pico_sdk_import.cmake .

copy ${env:PICO_EXAMPLES_PATH}\.vscode . -
  recurse
```

Al ejecutar estos comandos correctamente, la carpeta del proyecto contendrá el archivo `pico_sdk_import.cmake` y la subcarpeta `.vscode` con la configuración para Visual Studio Code.

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
.vscode	●	23/02/2026 11:52 a. m.	Carpeta de archivos	
build	●	19/05/2026 09:15 p. m.	Carpeta de archivos	
CMakeLists.txt	●	23/02/2026 12:49 p. m.	Documento de texto	1 KB
Parpadeo.c	●	23/02/2026 12:55 p. m.	Archivo de origen C	1 KB
pico_sdk_import.cmake	●	14/06/2023 04:31 a. m.	Archivo de origen C	4 KB

Figura 1: Estructura inicial de la carpeta **Práctica_4** tras copiar los archivos base del SDK.

II-B Creación del Proyecto

II-B1. Archivo CMakeLists.txt

El archivo `CMakeLists.txt` es el corazón del sistema de compilación CMake. Define el nombre del proyecto, las fuentes a compilar y las librerías que se deben enlazar. Para esta práctica, su contenido es el siguiente:

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_4)

pico_sdk_init()

add_executable(Parpadeo
  Parpadeo.C
)

target_link_libraries(Parpadeo
  pico_stdlib
  pico_cyw43_arch_none
)

pico_add_extra_outputs(Parpadeo)
```

Los campos más relevantes son:

- `set(PICO_BOARD pico_w)`: indica al compilador que la placa objetivo es la versión W, lo que habilita el soporte para el chip CYW43439.
- `add_executable`: especifica el nombre del ejecutable (`Parpadeo`) y el archivo fuente (`Parpadeo.c`).
- `target_link_libraries`: enlaza las librerías necesarias con el ejecutable.
- `pico_add_extra_outputs`: genera los archivos de salida adicionales, incluido el `.uf2` necesario para programar la placa.

II-B2. Implementación del código en C

Una vez configurado el sistema de compilación, se crea el archivo `Parpadeo.c` con el siguiente código,

que implementa el control del LED integrado mediante un bucle infinito:

```
CÓDIGO C – PARPADEO DEL LED INTEGRADO

#include "pico/stdlib.h"
#include "pico/cyw43_arch.h"

int main () {

    stdio_init_all();
    if (cyw43_arch_init()) {
        printf("WiFi init failed");
        return -1;
    }
    while(true) {
        cyw43_arch_gpio_put (
            CYW43_WL_GPIO_LED_PIN, 1);
        sleep_ms(250);

        cyw43_arch_gpio_put (
            CYW43_WL_GPIO_LED_PIN, 0);
        sleep_ms(250);
    }
}
```

La lógica del programa es la siguiente:

- `stdio_init_all()`: inicializa la comunicación estándar de entrada/salida (USB o UART), permitiendo usar `printf` para depuración.
- `cyw43_arch_init()`: inicializa el chip CYW43439. Si falla, el programa imprime un mensaje de error y termina.
- El bucle `while(true)` ejecuta indefinidamente el ciclo de encendido y apagado del LED con un intervalo de 250 ms en cada estado, produciendo un parpadeo a 2 Hz.
- `cyw43_arch_gpio_put (CYW43_WL_GPIO_LED_PIN, 1/0)`: establece el nivel lógico del GPIO al que está conectado el LED (alto = encendido, bajo = apagado).

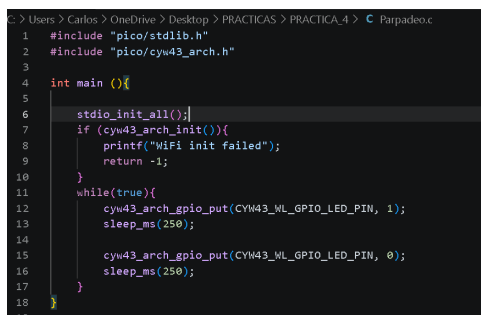


Figura 2: Código `Parpadeo.c` visualizado en Visual Studio Code con la extensión Pico.

II-C Proceso de Compilación y Carga del Programa

II-C1. Compilación en Visual Studio Code

Con el proyecto completamente configurado, se abre la aplicación **Pico Visual Studio Code**. Dentro del editor se realiza la siguiente secuencia:

1. **Abrir la carpeta del proyecto:** mediante *File* → *Open Folder*, se selecciona la carpeta `Práctica_4`.
2. **Confiar en el autor:** VS Code solicita confirmación de confianza en los archivos; se selecciona *Yes, I trust the authors*.
3. **Seleccionar el kit de compilación:** se elige *Pico ARM GCC* en el menú de selección de kits de CMake.
4. **Compilar el ejecutable:** en el panel de CMake (*Project Outline*), se posiciona el cursor sobre *Parpadeo (Executable)* y se presiona el ícono de compilación (*Build*).

El proceso de compilación puede observarse en la terminal integrada de VS Code. Una compilación exitosa genera el mensaje `Build finished with exit code 0` y crea la carpeta `build` dentro del directorio del proyecto.

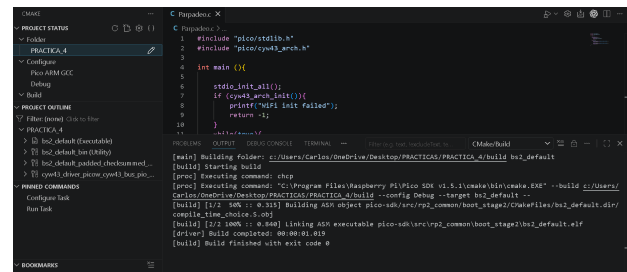


Figura 3: Terminal de Visual Studio Code mostrando la compilación exitosa del proyecto.

II-C2. Generación del archivo `.uf2` y transferencia al Pico W

Dentro de la carpeta `build` se generan múltiples archivos de salida. El archivo de interés es `Parpadeo.uf2`, que es el formato estándar para programar placas Raspberry Pi Pico mediante arrastrar y soltar (*drag and drop*).

El procedimiento de carga al microcontrolador es:

1. Mantener presionado el botón **BOOTSEL** de la Raspberry Pi Pico W.
2. Conectar el cable micro USB al ordenador sin soltar el botón.
3. Soltar el botón **BOOTSEL**. La placa aparecerá como un dispositivo USB de almacenamiento masivo llamado `RPI-RP2`.

4. Arrastrar el archivo `Parpadeo.uf2` a la unidad `RPI-RP2`.
5. La placa se reinicia automáticamente y comienza a ejecutar el programa.

II-C3. Verificación del funcionamiento

Tras la transferencia del archivo `.uf2`, el LED integrado de la Raspberry Pi Pico W comienza a parpadear de manera continua con un período de 500 ms (encendido 250 ms, apagado 250 ms), confirmando el correcto funcionamiento del programa.

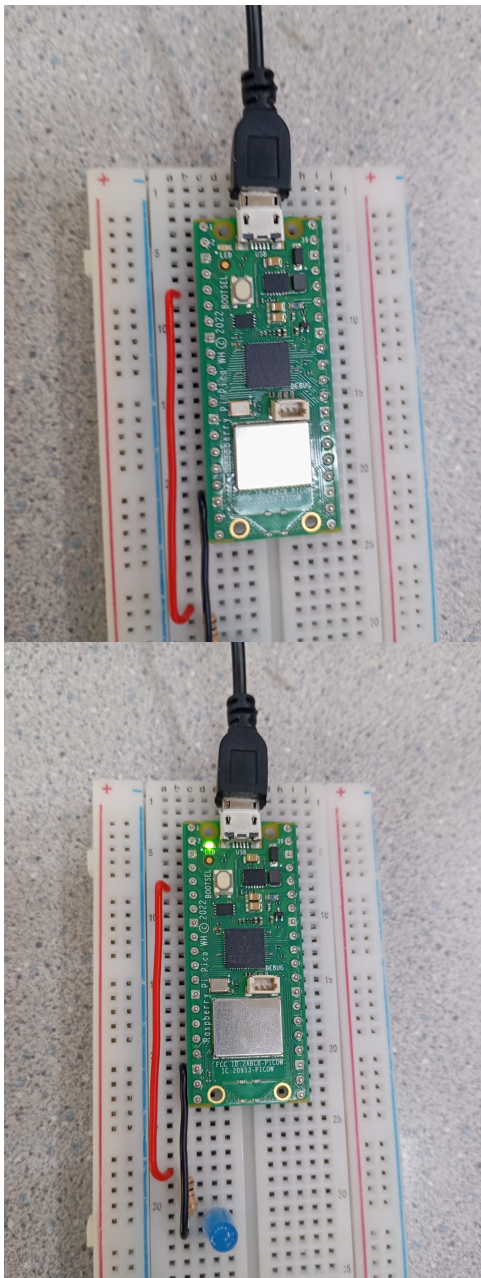


Figura 4: Raspberry Pi Pico W con el LED integrado encendido durante la fase de parpadeo.

III. RESULTADOS

III-A Evidencia del código utilizado

El código implementado en `Parpadeo.c` fue compilado sin errores ni advertencias usando el toolchain `arm-none-eabi-gcc` incluido en el SDK oficial de Raspberry Pi. La compilación generó los archivos de la Tabla I.

Tabla I: Archivos generados en la carpeta `build` tras la compilación

Archivo	Descripción
<code>Parpadeo.elf</code>	Ejecutable ELF para depuración
<code>Parpadeo.uf2</code>	Binario para programar la placa
<code>Parpadeo.bin</code>	Imagen binaria plana
<code>Parpadeo.hex</code>	Formato Intel HEX
<code>Parpadeo.elf.map</code>	Mapa de memoria del programa
<code>Parpadeo.dis</code>	Desensamblado del código

III-B Capturas del proceso

Las figuras 1, 2, 3, y 4 documentan cada etapa del proceso: configuración del entorno, escritura del código, compilación, modo de programación del dispositivo y verificación del funcionamiento, respectivamente.

III-C Errores encontrados y soluciones

Durante el desarrollo se identificaron los siguientes posibles puntos de falla y sus soluciones:

- **Error: `cyw43_arch_init()` retorna error.** Causa: no se especificó `set(PICO_BOARD pico_w)` en el `CMakeLists.txt`. Solución: agregar dicha línea y recompilar.
- **El LED no parpadea al cargar el `.uf2`.** Causa: el botón `BOOTSEL` no se mantuvo presionado durante la conexión USB. Solución: repetir el procedimiento de conexión correctamente.
- **VS Code no detecta el kit Pico ARM GCC.** Causa: la aplicación utilizada fue VS Code genérico en lugar de *Pico Visual Studio Code*. Solución: abrir la versión correcta del editor o instalar la extensión `CMake Tools`.

IV. PREGUNTAS DE REPASO

IV-A ¿De qué se encarga la librería `pico_cyw43_arch_none`?

Según la documentación oficial del SDK, la librería `pico_cyw43_arch_none` se encarga de inicializar y gestionar el chip CYW43439 integrado en la Raspberry Pi Pico W, sin activar ningún stack de red (de ahí el sufijo `none`). Esto permite acceder a los recursos de hardware del CYW43439, como el GPIO al que está conectado el LED integrado (`CYW43_WL_GPIO_LED_PIN`), sin la sobrecarga de inicializar Wi-Fi ni Bluetooth. Es la opción más ligera cuando el proyecto no requiere conectividad inalámbrica.

IV-B ¿Qué es el formato UF2?

UF2 (*USB Flashing Format*) es un formato de archivo binario diseñado específicamente para facilitar la programación de microcontroladores a través de una interfaz USB de almacenamiento masivo. Fue desarrollado por Microsoft y adoptado por la comunidad de Raspberry Pi. Su principal ventaja es que permite cargar un programa al microcontrolador simplemente arrastrando el archivo `.uf2` a la unidad USB que emula el dispositivo en modo bootloader, sin necesidad de programadores externos, drivers especiales ni software de flasheo adicional. Internamente, cada bloque de 512 bytes del archivo contiene una cabecera con información de dirección de memoria, lo que permite al bootloader escribir el firmware en las posiciones correctas de la memoria flash del microcontrolador.

IV-C ¿Cuáles son los voltajes de salida de los GPIO del Raspberry Pi Pico W?

Los pines GPIO del Raspberry Pi Pico W operan a un nivel lógico de 3.3V. En estado alto (lógico 1), los pines GPIO generan una tensión de salida de 3.3V, mientras que en estado bajo (lógico 0) la tensión es de 0V. Es importante destacar que estos pines **no son tolerantes a 5V**, por lo que conectar señales de 5V directamente a los GPIO puede dañar permanentemente el microcontrolador. La corriente máxima permitida por GPIO es de 12 mA, aunque se recomienda no superar los 4 mA para garantizar la estabilidad del sistema.

V. CONCLUSIONES

- La Raspberry Pi Pico W, programada en C mediante el SDK oficial, permite desarrollar aplicaciones embebidas de manera estructurada y eficiente. El proceso de configuración del entorno, aunque requiere varios pasos iniciales, es reproducible y bien documentado.
- El uso del archivo `CMakeLists.txt` resulta fundamental para la gestión del proyecto. Una configuración incorrecta de este archivo, como omitir `set(PICO_BOARD pico_w)`, impide el acceso a las librerías del chip CYW43439 y por tanto al LED integrado.
- La librería `pico_cyw43_arch_none` es indispensable en la Pico W para controlar el LED integrado, a diferencia de la Pico estándar donde basta con `pico_stdlib`. Esto evidencia la importancia de conocer las diferencias de hardware entre versiones de una misma plataforma.
- El formato UF2 simplifica enormemente el proceso de programación del microcontrolador al eliminar la necesidad de herramientas externas, haciendo al Raspberry Pi Pico W una plataforma muy accesible para el aprendizaje y prototipado rápido.
- Como mejora futura, se podría parametrizar el intervalo de parpadeo mediante una variable en lugar de usar valores fijos de 250 ms, facilitando ajustes sin modificar múltiples líneas del código. Igualmente, se podría agregar comunicación UART para monitorear el estado del LED en tiempo real desde una terminal serial.